

Security Audit

Immutable Ratings (Voting Platform)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	19
Conclusion	20
Our Methodology	21
Disclaimers	23
About Hashlock	24

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Immutable Ratings team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Immutable Ratings is a decentralized on-chain reputation protocol that empowers Web3 communities to instantly cast thumbs-up or thumbs-down across major social platforms—X, Warpcast, LinkedIn, and Wikipedia—while immutably storing every vote on the Base blockchain via smart contracts. By aggregating trustless feedback into real-time feeds, trending indexes, and community-driven leaderboards (Highest Rated, Lowest Rated, Trending, Newly Added), Immutable Ratings delivers transparent, tamper-proof social signals and reputation scores. Whether you're a content creator, DAO member, or brand, you can leverage these verifiable metrics to fuel decentralized governance, on-chain credit scoring, and next-generation social analytics—ushering in a new era of Web3 credibility and community-powered discovery.

Project Name: Immutable Ratings

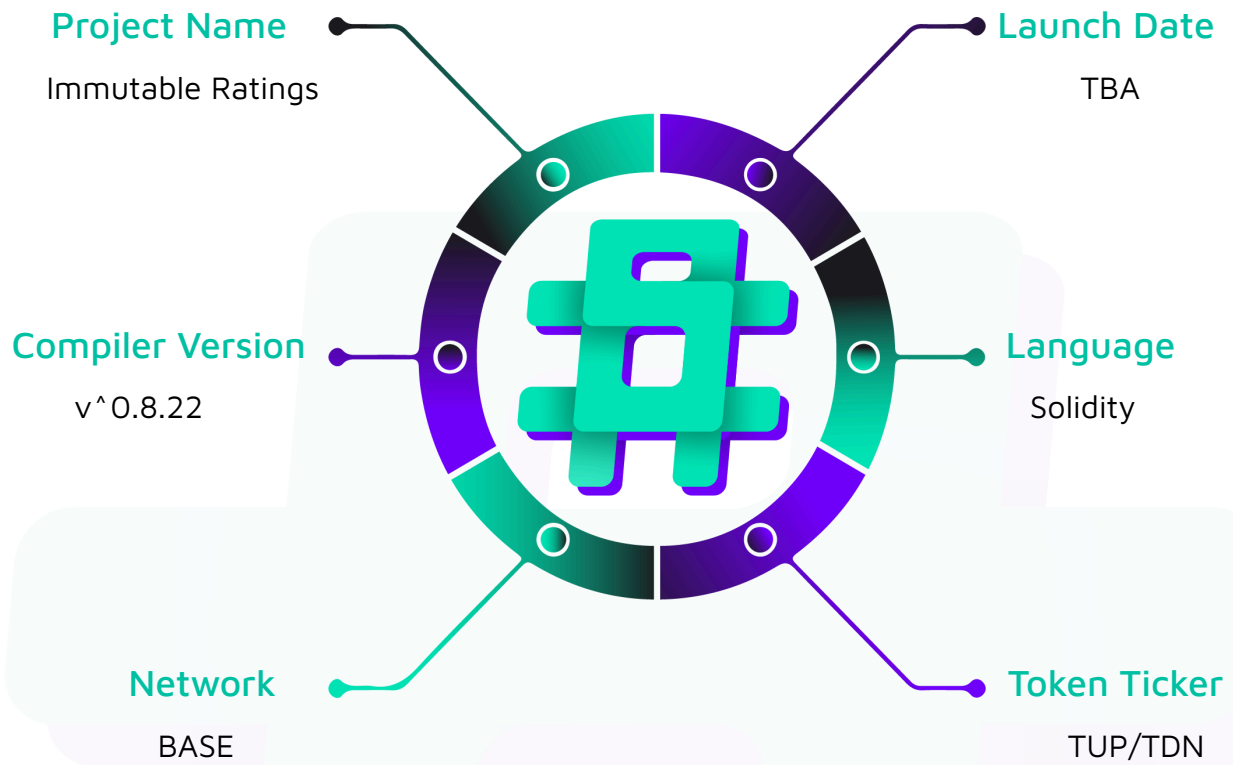
Project Type: Voting Platform

Compiler Version: ^0.8.22

Website: <https://www.ratings.wtf/>

Logo:



Visualised Context:

Project Visuals:

Immutable Ratings
Instantly rate with a THUMBS UP or THUMBS DOWN, securely stored forever on the Bloxis Blockchain. Now supporting Ethereum, Bitcoin, and Monero.

[Highest Rated](#) [Lowest Rated](#) [Trending](#) [Newest](#)

Search or enter a supported URL

HIGHEST RATED
Highest overall rating (up - down)

TRENDING
Trending over the last 7 days

LOWEST RATED
Lowest overall rating (up - down)

NEWLY ADDED
Newly added links

LEADERBOARD
Ranked by number of Immutable Ratings (IR)

Rank	Profile	IRs
#1	Ox40533...de789	50,000,000 IRs
#2	Ox70536...Ca8c6	145,843 IRs
#3	Ox4415b...e1C14	79,000 IRs
#4	morano.eth	71,838 IRs
#5	gnzlm.eth	23,000 IRs
#6	Ox5CcC6...92819	20,734 IRs
#7	Ox6A54B...B351a	15,000 IRs
#8	OxfC664...Ec11E	9,000 IRs

© 2025 Decentralized Ratings

#hashlock.

Hashlock Pty Ltd

Audit Scope

We at Hashlock audited the solidity code within the Immutable Ratings project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Immutable Ratings Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	April, 2025
Contract 1	ImmutableRatings.sol
Contract 1 MD5 Hash	057a6c4ec32c2174a55b53ba56e1ea4c
Contract 2	TDN.sol
Contract 2 MD5 Hash	fb930db8f0de12d69a3a2d7b7ace94d3
Contract 3	TUP.sol
Contract 3 MD5 Hash	36555303bf08b52979a16a3079c56af6
Audited GitHub Commit Hash	77672b903754094ac0a48339f5a40e9f75d446d6
Fix Review GitHub Commit Hash	65d2d0612d265c8f7fd8ca399c52ef7ddff72f14

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

2 Medium severity vulnerabilities

3 Low severity vulnerabilities

2 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
ImmutableRatings.sol - Allows users to: - Create a Market for a URL - Upvote a Market (Create an Up Rating) - Downvote a Market (Create a Down Rating) - Batch Upvote/Downvote Multiple Markets - View a User's Total Ratings - Preview the Cost of a Rating - Allows admins to: - Set the Fee Receiver - Pause/Unpause the Contract - Distribute ETH Payments to the Receiver	Contract achieves this functionality.
TDN.sol - Mint TDN tokens to a specified address (only by minters) - Track the number of upvotes (minted tokens) per user - Grant and manage access control roles (DEFAULT_ADMIN_ROLE, MINTER_ROLE)	Contract achieves this functionality.
TUP.sol - Mint TUP tokens to a specified address (only by minters) - Track the number of upvotes (minted tokens) per user - Grant and manage access control roles (DEFAULT_ADMIN_ROLE, MINTER_ROLE)	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Immutable Ratings project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Immutable Ratings project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] ImmutableRatings#createRatings - Potential Denial of Service in createRatings Function

Description

The `createRatings` function processes `upRatings` and `downRatings` arrays in two separate loops without enforcing a maximum array size. This unbounded iteration can consume excessive gas, potentially exceeding the block gas limit (e.g., Ethereum's ~30M gas limit). Malicious or uninformed users could submit large arrays (e.g., thousands of ratings), causing transactions to revert due to out-of-gas errors.

For example, A user submits 10,000 ratings in a single transaction, causing the transaction to consume >30M gas and revert, blocking legitimate batch rating submissions. Each loop iteration performs validation (`_validateRating`), market resolution (`_getMarket`), and token minting (`tokenUp.mint` or `tokenDown.mint`), all of which are gas-intensive. For large arrays, gas costs scale linearly, risking transaction failure. No checks exist to limit `upRatings.length` or `downRatings.length`.

Impact

This leads to transaction reverts and prevents users from submitting batch ratings, degrading platform usability too.

Recommendation

Introduce a `MAX_BATCH_SIZE` constant (e.g., 100) and enforce it with require checks at the function's start.

Status

Resolved

Medium

[M-01] ImmutableRatings#createRatings - Duplicate Market Submissions in Batch Rating Creation

Description

The `createRatings` function allows multiple ratings for the same market within `upRatings` or `downRatings` arrays, as no duplicate checks are implemented. The function's documentation implies it is designed for rating unique markets, but users can include the same market multiple times in a single transaction, leading to disruption to game logic.

Vulnerability Details

The loop processing `upRatings` and `downRatings` calls `_createUpRating` or `_createDownRating` without verifying if the `rating.market` (derived from `rating.url`) has already been processed in the same batch. This allows multiple `tokenUp.mint` or `tokenDown.mint` calls for the same market, inflating ratings and potentially skewing market scores.

For example, a user submits `upRatings` with 10 entries for the same market (same URL), resulting in 10x the intended upvote impact in one transaction.

Impact

Duplicate submissions could manipulate market ratings

Recommendation

Use a temporary mapping to track processed markets within a transaction and reject duplicates.

Status

Resolved

[M-02] ImmutableRatings#_distributePayment - Missing Refund Mechanism for Excess ETH leads to loss for the user.

Description

The `_distributePayment` function forwards the entire `msg.value` to the receiver address without refunding excess ETH if `msg.value` exceeds the required amount (`_getRatingPrice(totalUp + totalDown)`).

The function checks `msg.value >= _getRatingPrice` but does not calculate or return the difference. Excess ETH is irrevocably sent to the receiver, even if sent accidentally. For example, if the required price is 0.01 ETH but the user sends 1 ETH, the full 1 ETH is transferred.

Impact

Users lose excess funds, leading to financial loss and a poor user experience.

Recommendation

Calculate and refund excess ETH before transferring the required amount.

Status

Resolved

Low

[L-01] ImmutableRatings - Missing Two-Step Ownership change

Description

The contract uses `Ownable`, allowing single-step ownership transfers. A mistake in calling `transferOwnership` could transfer control to an incorrect or malicious address without confirmation.

Recommendation

Use `Ownable2Step` for a two-step ownership transfer process.

Status

Resolved

[L-02] ImmutableRatings - No Function to Recover Accidentally Sent Tokens

Description

The contract lacks a mechanism to recover ERC20 tokens accidentally sent to it – let it be ether or TUP/TDN.

Recommendation

Add a recovery function for recovery of ERC20/native tokens to either the owner address or any address as per convenience

Status

Resolved

[L-03] ImmutableRatings - Tokens were immutable upon deployment

Description

The TUP and TDN token addresses are immutable, preventing updates if tokens need upgrading.

Recommendation

Implement functions to change the values of the tokenUp and tokenDown variables.

Status

Acknowledged

QA

[Q-01] ImmutableRatings - Missing Event Emissions for Critical State Changes

Description

The contract does not emit events for state changes like receiver updates or isPaused toggles, hindering off-chain tracking.

Recommendation

Add events for state changes like setReceiver() and setIsPaused()

Status

Resolved

[Q-02] ImmutableRatings - Multiple Voting by Same user**Description**

The contract allows users to vote multiple times for the same market, which may be intentional but could enable rating manipulation.

Recommendation

If unintended, track user votes per market with a mapping.

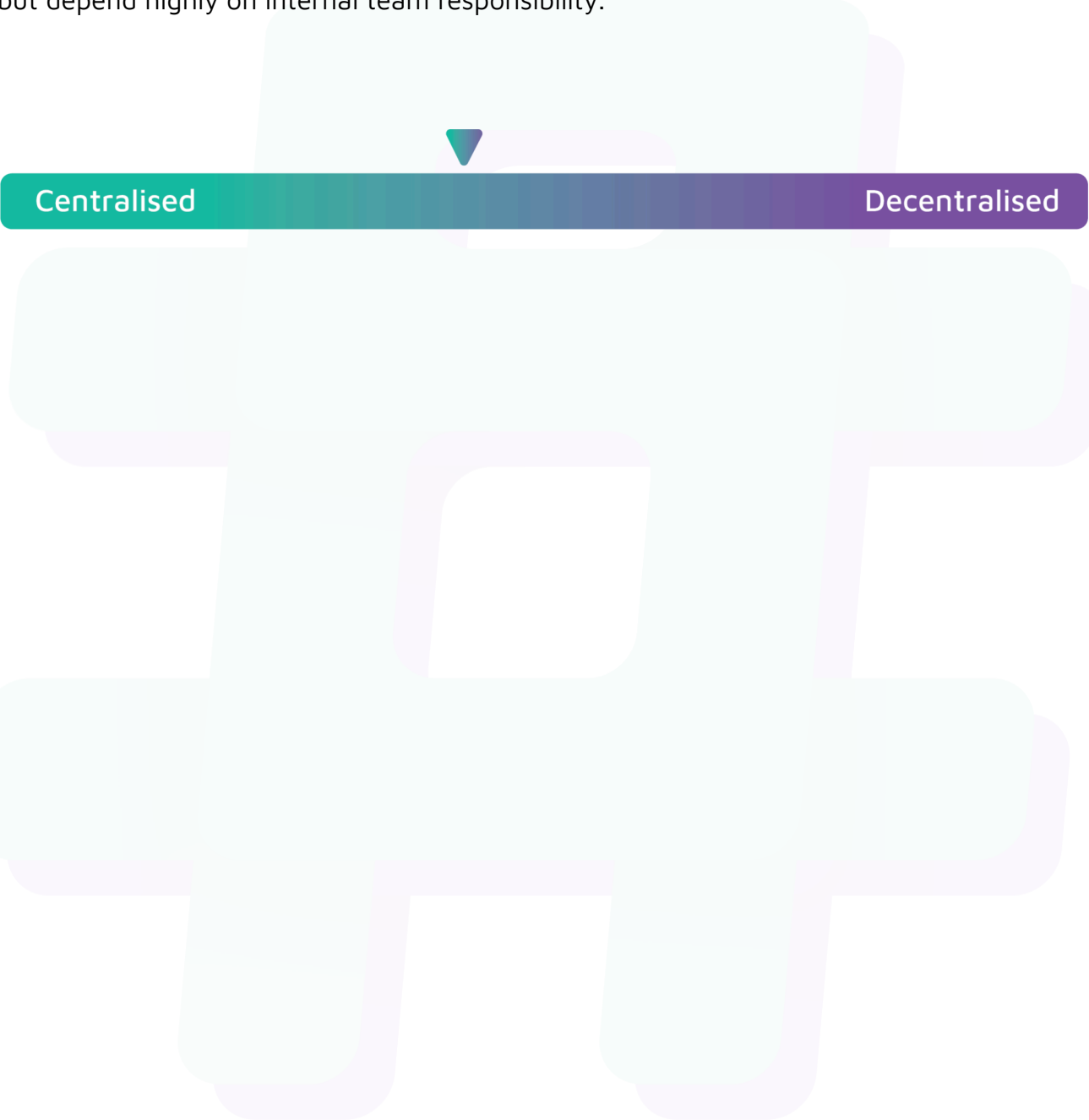
Status

Acknowledged

Centralisation

The Immutable Ratings project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Immutable Ratings project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.